

Contents

01 - はじめに	2
必要環境	3
ビルド手順	3
プロジェクトの初期化	3
最初のプログラム	3
コメントの書き方	3
02 - 変数と型	4
let による定数宣言	4
var による変数宣言	4
型アノテーション	4
基本型	4
文字列操作	5
エスケープシーケンス	5
再代入のルール	5
タプル分割代入	6
03 - 演算子	6
算術演算子	6
比較演算子	7
論理演算子	7
ビット演算子	8
複合代入演算子	8
インクリメント・デクリメント演算子	8
型昇格ルール	9
所属演算子	9
制御構文	10
if / elif / else	10
while ループ	10
for ループと range	10
break と continue	11
ネストの例	12
スコープのルール	12
match	13
関数	14
基本的な関数定義	14
関数の呼び出し	14
再帰関数	14
関数オーバーロード	15
戻り値型の省略 (Unit 型)	15
構造体と列挙型	15
record による構造体定義	15
コンストラクタの使い方	15
フィールドアクセス (ドット記法)	16
フィールドへの代入	16
関数の引数としての構造体	16
ネスト構造体	16
列挙型 (enum)	17
コレクション	17

タプル	17
リスト	18
マップ	20
セット	21
高度な機能	22
ラムダ関数	22
クロージャ	23
高階関数	23
関数を値として扱う	23
UFCS (Uniform Function Call Syntax)	24
演算子オーバーロード	24
Option 型	25
F-String (文字列補間)	25
型キャスト (as)	25
関連データを持つ enum (ADT)	26
ジェネリック enum	26
Result 型	26
パッケージ	27
from/import 構文	27
サブディレクトリ (ドット区切り)	27
ディレクトリパッケージ	27
標準ライブラリ (std)	28
検索パスの優先順位	28
PYTHONPATH 環境変数	28
制限事項	28
契約による設計	29
事前条件 (require)	29
事後条件 (ensure)	29
require と ensure の併用	29
構造体不変条件 (invariant)	30
ルール	30
テスト	30
テストの実行	30
テストの書き方	31
マッチャー	31
出力フォーマット	31
モック	32
パラメタライズドテスト	32
プロパティベーステスト	32
制限事項	33

[English](#) | [日本語](#) | [繁體中文](#)

01 - はじめに

次のチュートリアル → [02 - 変数と型](#)

必要環境

Ry をビルドして実行するには以下が必要です。

- LLVM 21
 - CMake 3.20 以上
 - C++17 対応コンパイラ (GCC 7+ / Clang 5+ 等)
-

ビルド手順

リポジトリのルートで以下のコマンドを実行します。

```
cmake -B build -DLLVM_DIR=/usr/local/llvm/lib/cmake/llvm
cmake --build build
```

ビルドが成功すると build/ry という実行ファイルが生成されます。

プロジェクトの初期化

ry init コマンドで新しいプロジェクトを作成できます。

```
mkdir my-project
cd my-project
ry init
```

これにより以下のファイルとディレクトリが生成されます。

- ry.toml — プロジェクト設定ファイル
- src/main.ry — エントリーポイント (サンプルコード付き)
- test/ — テストコード用ディレクトリ

詳細は [プロジェクト管理](#) を参照してください。

最初のプログラム

以下の内容を hello.ry というファイルに保存してください。

```
print("Hello, World!")
```

次のコマンドで実行します。

```
./build/ry hello.ry
```

出力:

```
Hello, World!
```

コメントの書き方

から行末までがコメントとして扱われます。

```
# これはコメントです
print("Hello") # 行末コメントも使えます
```

コメントはコードの動作に影響を与えません。

次のチュートリアル → [02 - 変数と型](#)

[English](#) | [日本語](#) | [繁體中文](#)

02 - 変数と型

← [01 - はじめに](#) / 次 → [03 - 演算子](#)

let による定数宣言

let キーワードで不変な変数（定数）を宣言します。型は右辺の値から自動的に推論されます。宣言後に値を変更することはできません。

```
let x = 42           # int 型として推論
let y = 3.14         # float 型として推論
let flag = true      # bool 型として推論
let name = "Ry"      # str 型として推論
```

var による変数宣言

var キーワードで可変な変数を宣言します。宣言後に同じ型の値を再代入できます。

```
var count = 0
count = count + 1  # OK: 同じ型への再代入
```

型アノテーション

変数の型を明示的に指定できます。

```
let x: int = 42
let rate: float = 0.5
let ok: bool = false
let msg: str = "hello"
```

型アノテーションと実際の値の型が一致しない場合はコンパイルエラーになります。

基本型

型	説明	リテラル例
int	64 ビット整数	0, 42, -10
byte	符号なし 8 ビット整数 (0-255)	let b: byte = 42
float	64 ビット浮動小数点数	0.0, 3.14, -1.5
bool	真偽値	true, false
str	文字列	"hello", ""

文字列操作

文字列に対してさまざまな操作が使えます。

```
let a = "Hello"
let b = "World"

# 結合
let c = a + ", " + b    # "Hello, World"

# 比較 (辞書順)
print(a == b)    # false
print(a != b)    # true
print(a < b)     # true ("H" < "W")

# 長さ
print(len(a))    # 5

# 部分文字列チェック
let s = "Hello, World!"
print(contains(s, "World"))    # true
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

エスケープシーケンス

文字列中で以下のエスケープシーケンスが使えます。

シーケンス	意味
\n	改行
\t	タブ
\\	バックスラッシュ
\"	ダブルクォート
\0	ヌル文字

```
print("Hello\nWorld")    # 2行に分けて出力
print("A\tB")            # タブ区切り
print("say \hi")         # ダブルクォートを含む文字列
```

再代入のルール

var で宣言した変数は再代入できます。ただし、以下の制限があります。

```
var x = 10
x = 20    # OK: 同じ型への再代入
# x = "text" # エラー: 型を変更する再代入は禁止
```

let は再代入できません。

```
let N = 5
# N = 6 # エラー: let 変数への再代入は禁止
```

同名の変数を再宣言することもできません。

```
let x = 1
# let x = 2 # エラー: 同名の再宣言は禁止
```

タプル分割代入

let または var を使って、タプルを複数の変数に一度に展開できます。

```
let a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

ワイルドカード

_ を使って特定の位置の値を無視できます。

```
let x, _ = (1, 2)    # x のみが束縛される; 2 は破棄
print(x)            # 1
```

可変変数での分割代入

var を使うと可変変数として宣言できます。

```
var a, b = (10, 20)
a = 99
print(a)    # 99
```

ルール

- 左辺の変数の数はタプルの要素数と一致させる必要があります。
- 各変数は通常の let/var 宣言と同じルールに従います。
- ネストされたタプルの分割代入はサポートされていません。

[← 01 - はじめに](#) / [次 → 03 - 演算子](#)

[English](#) | [日本語](#) | [繁體中文](#)

03 - 演算子

[← 02 - 変数と型](#) / [次 → 04 - 制御構文](#)

算術演算子

演算子	説明	例	結果
+	加算	3 + 2	5
-	減算	3 - 2	1
*	乗算 / 文字列繰り返し	3 * 2	6
/	除算 (常に float)	7 / 2	3.5
//	整数除算 (常に int)	7 // 2	3
%	剰余	7 % 3	1
**	累乗 (常に float)	2 ** 10	1024.0

```

let a = 10
let b = 3

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256.0 (float)

```

比較演算子

比較演算子はすべて bool 値を返します。

演算子	説明	例
==	等しい	a == b
!=	等しくない	a != b
<	より小さい	a < b
<=	以下	a <= b
>	より大きい	a > b
>=	以上	a >= b

```

let x = 5
let y = 10

print(x == y)    # false
print(x != y)    # true
print(x < y)     # true
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false

```

文字列にも比較演算子が使えます（辞書順比較）。

```

print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true

```

論理演算子

演算子	説明	例
and	論理 AND	a and b
or	論理 OR	a or b
not	論理 NOT	not a

```

let t = true
let f = false

print(t and f)    # false

```

```
print(t or f)    # true
print(not t)     # false
print(not f)     # true
```

ビット演算子

ビット演算子は `int` 型にのみ使用できます。

演算子	説明	例
<code>&</code>	ビット AND	<code>5 & 3 → 1</code>
<code> </code>	ビット OR	<code>5 3 → 7</code>
<code>^</code>	ビット XOR	<code>5 ^ 3 → 6</code>
<code>~</code>	ビット NOT (単項)	<code>~5 → -6</code>
<code><<</code>	左シフト	<code>1 << 3 → 8</code>
<code>>></code>	算術右シフト	<code>8 >> 2 → 2</code>
<code>>>></code>	論理右シフト	<code>-1 >>> 1 → 9223372036854775807</code>

```
let a = 0b1010    # 10
let b = 0b1100    # 12

print(a & b)      # 8 (0b1000)
print(a | b)      # 14 (0b1110)
print(a ^ b)      # 6 (0b0110)
print(~a)         # -11
print(1 << 4)     # 16
print(32 >> 2)    # 8
```

複合代入演算子

変数の値を更新する際に使える省略記法です。

演算子	説明	同等の式
<code>+=</code>	加算代入	<code>x = x + n</code>
<code>-=</code>	減算代入	<code>x = x - n</code>
<code>*=</code>	乗算代入	<code>x = x * n</code>
<code>/=</code>	除算代入	<code>x = x / n</code>
<code>%=</code>	剰余代入	<code>x = x % n</code>

```
let x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6.0 (float になる)
```

インクリメント・デクリメント演算子

変数を 1 増減させるための省略記法です。

演算子	説明	同等の式
x++	1 を加算	x = x + 1
x--	1 を減算	x = x - 1

```
var count = 0
count++      # count == 1
count++      # count == 2
count--      # count == 1
```

注意: ステートメントとしてのみ使用可能で、式の中では使えません。

型昇格ルール

演算において int と float が混在する場合の挙動を以下に示します。

```
# + - * は片方が float なら結果は float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3.0 (float)
print(1.0 + 2)    # 3.0 (float)

# / は常に float
print(4 / 2)      # 2.0 (float)

# // は常に int
print(7 // 2)     # 3 (int)
print(7.0 // 2)   # 3 (int)

# ** は常に float
print(2 ** 3)     # 8.0 (float)

# % は両辺 int なら int、片方 float なら float
print(7 % 3)      # 1 (int)
print(7.5 % 2)    # 1.5 (float)

# + は両辺 str なら文字列結合
print("foo" + "bar") # "foobar"

# * は片方が str、もう片方が int なら文字列繰り返し
print("ab" * 3)    # "ababab"
print(3 * "ab")    # "ababab"
```

所属演算子

演算子	説明	例
in	所属チェック	2 in {1, 2, 3} → true
not in	否定の所属チェック	4 not in {1, 2, 3} → true

```
let s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true
```

← [02 - 変数と型](#) / 次 → [04 - 制御構文](#)

[English](#) | [日本語](#) | [繁體中文](#)

制御構文

← 前: [演算子](#) | 次: [関数](#) →

if / elif / else

条件に応じて処理を分岐させるには if を使います。

```
let x = 10
```

```
if x > 0:
    print(x)
elif x == 0:
    print(0)
else:
    print(-1)
```

- elif と else は省略できます。
- 条件式には bool 以外も指定できます。int の場合、0 が偽、非 0 が真として扱われます。
- if はネストできます。

```
let a = 5
let b = 3
```

```
if a > 0:
    if b > 0:
        print(a + b)    # 8
```

while ループ

条件が真である間、ブロックを繰り返し実行します。

```
let i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for ループと range

リストまたは range を使ってイテレーションできます。

```
for x in [1, 2, 3]:
    print(x)
# 1
```

```
# 2
# 3
```

range(n) は 0 から n - 1 までの整数を生成します。

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4
```

range(start, end) は start から end - 1 までの整数を生成します。

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

.. 範囲演算子は両端を含む範囲を生成します。1 .. 3 は [1, 2, 3] を生成します。

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

for k, v in map でマップのキーと値を走査できます。

```
let m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

break と continue

break はループを即座に抜けます。continue は現在の反復をスキップして次の反復へ進みます。

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

while でも同様に使用できます。

```
let n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
```

```
    print(n)
# 1
# 3
# 5
# 7
```

注意: ネストしたループの中では、`break` / `continue` は最も内側のループにのみ作用します。ループ外で使用するとコンパイルエラーになります。

ネストの例

`for` と `while` はネストできます。

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
# 11
# 13
# 21
# 23
# 31
# 33
```

スコープのルール

制御構文のブロックはスコープを持ちます。

ブロックスコープ

ブロック内で宣言した変数はブロック外から参照できません。

```
if true:
    let inner = 42
# ここで inner を参照するとコンパイルエラー
```

外側の変数への参照・再代入

ブロック内から外側の変数を参照・再代入できます。

```
let count = 0
for i in range(5):
    count = count + i
print(count)    # 10
```

シャドーイング

外側と同名の変数をブロック内で宣言すると、ブロック内ではその新しい変数が使われます（シャドーイング）。外側の変数は変化しません。

```
let x = 1
if true:
    let x = 99
```

```
    print(x)    # 99
print(x)        # 1
```

match

match は値に応じた分岐を行う構文です。enum や Option を安全に処理できます。

```
enum Color:
    Red
    Green
    Blue

let c = Color::Green
match c:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")
# green
```

Option のマッチ

match を使うことで、None の場合も安全に処理できます。

```
let x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")
# 42
```

ワイルドカードとリテラル

_ は何にでもマッチするワイルドカードパターンです。リテラル値（数値・文字列・真偽値）でもマッチできます。

```
let n = 5
match n:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")
# other
```

guard 節

if でガード条件を追加できます。

```
match n:
    case x if x > 0:
        print("positive")
```

```
case x if x < 0:
    print("negative")
case _:
    print("zero")
```

注意: match はすべてのパターンを網羅する必要があります。enum はすべてのバリエント、Option は Some と None の両方、リテラルは _ が必要です。

[← 前: 演算子](#) | [次: 関数](#) →

[English](#) | [日本語](#) | [繁體中文](#)

関数

[← 前: 制御構文](#) | [次: 構造体](#) →

基本的な関数定義

関数は fn キーワードで定義します。引数の型宣言は必須で name: type の形式で指定します。戻り値の型は -> の後に指定します。

```
fn add(a: int, b: int) -> int:
    return a + b
```

- 引数の型宣言は必須です。
- 戻り値型は -> の後に指定します。
- return 文で値を返します。

関数の呼び出し

定義した関数は名前と引数を指定して呼び出します。

```
fn multiply(x: int, y: int) -> int:
    return x * y

let result = multiply(3, 4)
print(result)    # 12
```

再帰関数

関数は自分自身を呼び出せます（再帰）。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120
print(factorial(0))    # 1
```

関数オーバーロード

引数の数や型が異なる同名の関数を複数定義できます。

```
fn add(a: int, b: int) -> int:
    return a + b

fn add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))          # 3
print(add(1.5, 2.5))      # 4
```

呼び出し時の引数の型に応じて、適切な関数が自動的に選択されます。

注意: 引数の型が同一で戻り値の型だけが異なる関数を定義するとコンパイルエラーになります。

戻り値型の省略 (Unit 型)

戻り値が不要な関数は `->` を省略できます。この場合、関数は `Unit` 型を返します。

```
fn greet():
    print(42)

greet()    # 42
```

引数なし・戻り値なしの最もシンプルな関数形式です。

[← 前: 制御構文](#) | [次: 構造体](#) →

[English](#) | [日本語](#) | [繁體中文](#)

構造体と列挙型

[← 前: 関数](#) | [次: コレクション](#) →

record による構造体定義

`record` キーワードで構造体を定義します。各フィールドは `name: type` の形式で記述します。

```
record Point:
    x: int
    y: int
```

構造体はスタック上の値型です。

コンストラクタの使い方

構造体名を関数のように呼び出してインスタンスを生成します。引数はフィールドの定義順に指定します。

```
let p = Point(10, 20)
```

フィールドアクセス（ドット記法）

フィールドにはドット記法でアクセスします。

```
let p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

注意: print に構造体を直接渡すとエラーになります。フィールドを個別に渡してください。

フィールドへの代入

var で宣言した変数のフィールドは再代入できます。

```
var p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

注意: let で宣言した変数のフィールドへの代入はコンパイルエラーになります。

関数の引数としての構造体

構造体を関数の引数として渡せます。

```
record Point:
  x: int
  y: int

fn distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

let p1 = Point(10, 3)
let p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

ネスト構造体

構造体のフィールドに別の構造体を使えます。

```
record Point:
  x: int
  y: int

record Line:
  start: Point
  end: Point

let line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

ドット記法をチェーンすることでネストしたフィールドにアクセスできます。

列挙型 (enum)

enum キーワードで列挙型を定義できます。各バリエントは名前付きの定数として扱われます。

定義

```
enum Color:
    Red
    Green
    Blue
```

使い方

バリエントには `::` でアクセスします。

```
let c = Color::Red
print(c)    # Red
```

比較

`==` や `!=` でバリエントを比較できます。

```
if c == Color::Red:
    print("red!")
elif c == Color::Green:
    print("green!")
else:
    print("blue!")
```

関数引数

関数の引数型として enum 名を使えます。

```
fn describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

[← 前: 関数](#) | [次: コレクション](#) →

[English](#) | [日本語](#) | [繁體中文](#)

コレクション

[← 前: 構造体](#) | [次: 高度な機能](#) →

Ry には 4 種類のコレクション型があります: **タプル**、**リスト**、**マップ**、**セット**。

タプル

タプルは複数の値を一つにまとめた不変のデータ構造です。異なる型の要素を保持できます。

生成

```
let t = (1, 3.14)
```

型アノテーション

```
let t: (int, float) = (1, 3.14)
```

要素アクセス

.0, .1, ... のようにインデックスでアクセスします。

```
let t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

関数の戻り値

複数の値を返したいときにタプルが便利です。

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
let result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

制限事項

- 範囲外のインデックス（例: 要素数2のタプルに .2 でアクセス）はコンパイルエラーになります。
 - print にタプルを直接渡すとエラーになります。各要素を個別に渡してください。
-

リスト

リストは同じ型の要素を並べた可変長のデータ構造です。

生成

```
let xs = [1, 2, 3]
```

型アノテーション

```
let xs: List<int> = [1, 2, 3]
```

インデックスアクセス

```
print(xs[0])    # 1
```

```
let i = 1
print(xs[i])    # 2
```

インデックス代入

```
xs[0] = 99
```

len

```
print(len(xs))  # 3
```

print

```
print(xs)    # [1, 2, 3]
```

for 走査

```
for x in xs:
    print(x)
```

関数引数

```
fn first(xs: List<int>) -> int:
    return xs[0]
```

filter, map, sort

リストは filter、map、sort 操作をサポートしています。これらは元のリストを変更せず、新しいリストを返します。

```
let xs = [1, 2, 3, 4, 5]
```

filter: 条件に一致する要素だけを残す

```
let evens = xs.filter(fn(x: int): x > 3)
print(evens)    # [4, 5]
```

map: 各要素を変換する

```
let doubled = xs.map(fn(x: int): x * 2)
print(doubled)    # [2, 4, 6, 8, 10]
```

sort: 昇順ソート (デフォルト)

```
let sorted = [3, 1, 2].sort()
print(sorted)    # [1, 2, 3]
```

チェーン

```
let result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce, fold

reduce はリストを最初の要素から 1 つの値に集約します。fold は明示的な初期値を指定できます。

```
let xs = [1, 2, 3, 4, 5]
```

reduce: 最初の要素から開始

```
let total = reduce(xs, fn(a: int, b: int): a + b)
print(total)    # 15
```

fold: 明示的な初期値を指定

```
let total2 = fold(xs, 0, fn(a: int, b: int): a + b)
print(total2)    # 15
```

any, all

any は述語を満たす要素が 1 つ以上あれば true を返します。all はすべての要素が満たす場合に true を返します。

```
let xs = [1, 2, 3, 4, 5]
```

```

print(any(xs, fn(x: int): x > 4))    # true
print(any(xs, fn(x: int): x > 9))    # false

print(all(xs, fn(x: int): x > 0))    # true
print(all(xs, fn(x: int): x > 3))    # false

```

sum, min, max

```

let xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5

```

first, last, is_empty

```

let xs = [10, 20, 30]
print(first(xs))    # 10
print(last(xs))     # 30
print(is_empty(xs)) # false

```

enumerate, zip

`enumerate` は各要素にインデックスを付けます。`zip` は2つのリストを要素ごとに結合します。

```

let xs = [10, 20, 30]
let indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

let ys = ["a", "b", "c"]
let zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]

```

制限事項

- 全要素が同じ型である必要があります。異なる型を混在させることはできません。
- 空リスト `[]` はエラーになります。
- 範囲外アクセスはランタイムエラー (`exit(1)`) になります。
- 要素の型として `int`, `float`, `bool`, `str` をサポートしています。

マップ

マップはキーと値のペアを管理する連想配列です。

生成

```
let m = {"a": 1, "b": 2}
```

型アノテーション

```
let m: Map<str, int> = {"a": 1, "b": 2}
```

キーアクセス

```
print(m["a"])    # 1
```

挿入 / 更新

新規キーへの代入で挿入、既存キーへの代入で更新します。

```
m["c"] = 3      # 新規追加  
m["a"] = 99     # 更新
```

len

```
print(len(m))   # 3
```

print

```
print(m)        # {a: 99, b: 2, c: 3}
```

has_key

キーが存在するか確認します。

```
print(m.has_key("a"))    # true
```

keys, values

keys は全キーのリストを返します。values は全値のリストを返します。

```
let m = {"a": 1, "b": 2, "c": 3}  
print(keys(m))    # ["a", "b", "c"]  
print(values(m))  # [1, 2, 3]
```

関数引数

```
fn get_val(m: Map<str, int>, k: str) -> int:  
    return m[k]
```

制限事項

- 全キーが同じ型、全値が同じ型である必要があります。
 - 空マップは型注釈が必要です。
 - 存在しないキーへのアクセスはランタイムエラー（exit(1)）になります。
-

セット

セットは同じ型の要素を重複なしで保持するコレクションです。

生成

```
let s = {1, 2, 3}
```

型アノテーション

```
let s: Set<int> = {1, 2, 3}
```

in 演算子

要素がセットに含まれるかを in 演算子で確認できます。

```
print(2 in s)    # true
print(5 in s)    # false
```

add / remove

```
s.add(4)         # 要素追加
s.remove(1)      # 要素削除
s.add(2)         # 既に存在するため無視
```

len / print

```
print(len(s))    # 3
print(s)         # {2, 3, 4}
```

for 走査

```
for x in s:
    print(x)
```

空セット

空セットは型注釈が必要です。

```
let empty: Set<int> = {}
```

制限事項

- 全要素が同じ型である必要があります。
- 要素の型として int, float, bool, str をサポートしています。

[← 前: 構造体](#) | [次: 高度な機能](#) →

[English](#) | [日本語](#) | [繁體中文](#)

高度な機能

[← 前: コレクション](#) | [次: パッケージ](#) →

ラムダ関数

ラムダ関数は、関数を式として記述する構文です。fn(引数): 式の形で書きます。戻り値型は自動推論されます。

単一式ラムダ

```
let double = fn(x: int): x * 2
print(double(5))    # 10

let add = fn(a: int, b: int): a + b
print(add(3, 4))    # 7
```

引数なしラムダ

```
let answer = fn(): 42
print(answer()) # 42
```

複数行ラムダ

: の後に改行してインデントすることで、複数の文を書けます。

```
let abs = fn(x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3)) # 3
```

クロージャ

ラムダ関数は定義時のスコープにある変数をキャプチャできます。

```
let offset = 10
let add_offset = fn(x: int): x + offset
print(add_offset(5)) # 15
```

高階関数

関数を引数として受け取る関数を定義できます。関数型は `fn(引数型) -> 戻り値型` と書きます。

```
fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

let double = fn(x: int): x * 2
print(apply(double, 3)) # 6
print(apply(fn(n: int): n + 1, 10)) # 11
```

関数を値として扱う

名前付き関数も変数に束縛したり、引数として渡したりできます。

```
fn square(x: int) -> int:
    return x * x

fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

# 名前付き関数を引数として渡す
print(apply(square, 4)) # 16

# 変数に束縛する
let sq = square
print(sq(5)) # 25
```

UFCS (Uniform Function Call Syntax)

UFCS を使うと、`f(a, b)` の呼び出しを `a.f(b)` と書けます。メソッドチェーンのような記述が可能になります。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
let x = 1
print(x.add(2))    # add(x, 2) → 3
```

チェーン呼び出し

```
fn double(n: int) -> int:
    return n * 2
```

```
print(x.add(2).double())    # double(add(x, 2)) → 6
```

演算子オーバーロード

`fn operator` 演算子 構文でカスタム型に演算子を定義できます。

二項演算子

パラメータを 2 個取ります。

```
record Vec2:
    x: int
    y: int
```

```
fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

```
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

```
let v1 = Vec2(1, 2)
let v2 = Vec2(3, 4)
let v3 = v1 + v2
print(v3.x)    # 4
print(v1 == v2)    # false
```

単項演算子

パラメータを 1 個取ります。

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

対応演算子一覧

種別	演算子
算術	+, -, *, /, %, **, //
比較	==, !=, <, <=, >, >=
ビット	&, \ , ^, ~, <<, >>
論理	and, or, not

Option 型

値が存在するかどうかを表す型です。Some(値) または None をとります。

```
let x: Option<int> = Some(42)
print(x)           # Some(42)

let y: Option<int> = None
print(y)           # None
```

値の取り出し

match を使って内部の値を安全に取り出し、None の場合も処理できます。

```
match x:
  case Some(v):
    print(v)        # 42
  case None:
    print("nothing")
```

F-String (文字列補間)

f"..." を使って、文字列内に式を直接埋め込むことができます。式は{} 内に記述します。

```
let name = "Alice"
print(f"Hello {name}")    # Hello Alice

let x = 3
let y = 4
print(f"{x} + {y} = {x + y}")    # 3 + 4 = 7
```

リテラルの波括弧を出力するには {{ と }} を使います。

```
print(f"{{escaped}}")    # {escaped}
```

型キャスト (as)

as を使って型を明示的に変換できます。

```
let x = 42 as float    # 42.0
let y = 3.14 as int    # 3 (切り捨て)
let s = 42 as str      # "42"
let b = true as int    # 1
```

関連データを持つ enum (ADT)

enum バリエントに関連する値を持たせることができます。これにより、1 つの enum でさまざまな形のデータファミリーを表現できます。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

ADT バリエントの構築

```
let c = Shape::Circle(3.14)
let r = Shape::Rectangle(4.0, 5.0)
let p = Shape::Point
```

ADT バリエントのマッチング

case にバインディングパターンを使って関連データを取り出します。

```
fn describe(s: Shape) -> str:
    match s:
        case Shape::Circle(r):
            return f"circle with radius {r}"
        case Shape::Rectangle(w, h):
            return f"rectangle {w}x{h}"
        case Shape::Point:
            return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0
```

ジェネリック enum

enum は型パラメータを取ることができ、異なるペイロード型で再利用可能になります。

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

使用法

```
let a = MyOption<int>::MySome(42)
let b: MyOption<int> = MyOption<int>::MyNone

match a:
    case MyOption::MySome(v):
        print(v)      # 42
    case MyOption::MyNone:
        print("none")
```

Result 型

Result<T, E> は失敗する可能性のある関数に使用します。成功時は Ok(value)、失敗時は Err(error) を返します。

```
fn divide(a: int, b: int) -> Result<int, str>:
    if b == 0:
        return Err("division by zero")
    return Ok(a // b)
```

match を使って結果を処理します。

```
let r = divide(10, 0)
match r:
    case Ok(v):
        print(v)
    case Err(e):
        print(e)    # division by zero
```

[← 前: コレクション](#) | [次: パッケージ](#) →

[English](#) | [日本語](#) | [繁體中文](#)

パッケージ

[← 前: 高度な機能](#) | [次: 契約による設計](#) →

Ry はパッケージシステムを使って、ファイルやディレクトリにまたがるコードを管理します。詳細な仕様は [パッケージリファレンス](#) を参照してください。

from/import 構文

別ファイルの関数をインポートするには from 構文を使います。

```
from math import add, sub    # 選択インポート
from math                    # 全関数インポート
```

これで math.ry に定義された関数が使えるようになります。

サブディレクトリ（ドット区切り）

ドット区切りでサブディレクトリ内のパッケージを指定できます。

```
from utils.math import add    # utils/math.ry をインポート
```

ドット 1 つがディレクトリの区切りに対応します。

ディレクトリパッケージ

パッケージは単一の .ry ファイルでも、複数の .ry ファイルを含むディレクトリでも構いません。パッケージがディレクトリに解決される場合、その中のすべての .ry ファイルが自動的にロードされます。

```
mypackage/
    math.ry      # fn add(), fn sub()
    string.ry    # fn concat()

from mypackage          # add, sub, concat をインポート
from mypackage import add # add のみインポート
```

特別なエントリファイル（`__init__.py` のようなもの）は不要です。 `_` で始まるファイルは除外されます。

標準ライブラリ（std）

std パッケージはすべてのプログラムに自動的にインポートされます。 `from std` を記述する必要はありません。

```
# これらの関数はインポートなしで利用可能
print("hello")
let n = len("world")
let xs = range(5)
```

std のサブパッケージから特定の定義を明示的にインポートすることもできます。

```
from std.str import contains
```

RY_HOME

標準ライブラリは `$RY_HOME/lib/std/` にインストールされます。 `RY_HOME` のデフォルト値は `~/.ry` です。

```
export RY_HOME="$HOME/.ry" # デフォルト
```

検索パスの優先順位

パッケージファイルは以下の順序で検索されます。

1. インポート元ファイルのディレクトリ — インポートを記述したファイルと同じディレクトリを最初に探します。
 2. `$RY_HOME/lib` — 標準ライブラリの場所。
 3. 実行ファイル相対の `lib/` — `ry` 実行ファイルからの相対ディレクトリ。
 4. `RY_PATH` 環境変数 — 見つからない場合は `RY_PATH` に指定されたディレクトリを順番に検索します。
-

RY_PATH 環境変数

複数のディレクトリをコロン区切りで指定できます。

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

設定後は、指定ディレクトリ内のパッケージをどこからでもインポートできます。

制限事項

- `from` 文はファイルのトップレベルにのみ記述できます。関数やブロックの内部には書けません。
- 同じパッケージを複数回インポートしても、自動的にスキップされます（二重インポートは発生しません）。
- 循環インポート（A が B をインポートし、B が A をインポートする）はエラーになります。

```
# エラー例: a.ry と b.ry が互いをインポートしている場合
# a.ry: from b import foo
# b.ry: from a import bar ← 循環インポートエラー
```

[← 前: 高度な機能](#) | [次: 契約による設計](#) →

[English](#) | [日本語](#) | [繁體中文](#)

契約による設計

[← 前: パッケージ](#) | [次: テスト](#) →

Ry は Eiffel スタイルの契約による設計をサポートしています。事前条件 (`require`)、事後条件 (`ensure`)、構造体不変条件 (`invariant`) を使ってコードの正しさを保証します。契約違反が発生するとプログラムは終了します。詳細な仕様は[契約による設計リファレンス](#)を参照してください。

事前条件 (`require`)

`require` を使って、関数が呼び出される際に真でなければならない条件を指定します。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  var new_balance: int = balance + amount
  return new_balance
```

事前条件が満たされない場合、以下のメッセージとともにプログラムが終了します。

```
Contract violation: require failed in deposit()
```

事後条件 (`ensure`)

`ensure` を使って、関数が値を返す際に真でなければならない条件を指定します。

`result` キーワード

`ensure` ブロック内では、`result` で戻り値を参照できます。

```
fn abs(x: int) -> int:
  ensure:
    result >= 0
  if x < 0:
    return -x
  return x
```

`old()` 式

`old(expr)` は関数開始時の式の値をキャプチャします。状態の変化前後を比較するのに便利です。

```
fn increment(x: int) -> int:
  ensure:
    result == old(x) + 1
  return x + 1
```

`require` と `ensure` の併用

両方を同時に使えます。`require` は `ensure` の前に記述する必要があります。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
  ensure:
```

```
    balance >= 0
  ensure:
    result >= 0
    result == old(balance) + amount
  var new_balance: int = balance + amount
  return new_balance
```

構造体不変条件 (invariant)

`invariant` を使って、構造体に対して常に成り立つ必要がある条件を指定します。不変条件はコンストラクタの実行後とフィールドへの代入後にチェックされます。

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

let a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1              # Contract violation: invariant failed
```

ルール

- `require` と `ensure` ブロックは任意で、関数本体の前に記述します。
 - 両方を使う場合、`require` を `ensure` の前に記述する必要があります。
 - `result` と `old()` は `ensure` ブロック内でのみ使用できます。
 - `invariant` は `record` 定義の末尾、全フィールド宣言の後に記述します。
 - すべての契約違反は `exit(1)` でプログラムを終了します。
-

[← 前: パッケージ](#) | [次: テスト](#) →

[English](#) | [日本語](#) | [繁體中文](#)

テスト

[← 前: 契約による設計](#)

Ry には `describe`、`it`、`expect` を使った RSpec スタイルの組み込みテスト構文があります。詳細な仕様は[テストリファレンス](#)を参照してください。

テストの実行

```
ry test                                # *.test.ry ファイルを自動検出して実行
ry test tests/my_test.test.ry         # 特定のテストファイルを実行
```

すべてのテストが成功すると終了コード 0、1 つでも失敗すると 1 が返されます。

引数なしで実行すると、`ry test` は `ry.toml` を探してプロジェクトルートを特定し、すべての `*.test.ry` ファイルを再帰的に検出します。

テストの書き方

`describe` で関連するテストをグループ化し、`it` で個々のテストケースを定義します。

```
describe("Calculator", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)

  )
  it("subtracts integers", fn():
    expect(5 - 3).to_eq(2)

  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()
  )
)
```

- `describe` と `it` は説明文字列とラムダ引数 `fn()` を第二引数に取ります。
- `describe` / `it` / `expect` / `mock` / `verify` は `ry test` でのみ使用できます（通常の `ry` 実行ではコンパイルエラー）。

マッチャー

マッチャー	説明	対応型
<code>to_eq(expected)</code>	等値比較	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	不等値アサーション	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> アサーション	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> アサーション	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> アサーション	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> が <code>Some</code> であるアサーション	<code>Option</code>
<code>to_contain(val)</code>	コンテナに値が含まれるアサーション	<code>List</code> , <code>Set</code> , <code>str</code>

出力フォーマット

```
Calculator
+ adds integers
+ subtracts integers
- checks booleans
  line 10: expected true, got false
```

2 passed, 1 failed

+ は成功（緑）、- は失敗（赤）を示します。

モック

`mock(fn_name, replacement)`

現在の `it` ブロック内で関数をモック実装に置き換えます。`it` ブロック終了時に自動的に復元されます。

```
fn fetch_data() -> str:
    return "real data"

describe("mocking", fn():
    it("replaces function", fn():
        mock(fetch_data, fn(): "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)
```

`verify(fn_name)`

モックされた関数の呼び出し回数を返します。

```
describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn(): "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

パラメタライズドテスト

`@each` を使って同じテストを複数の入力で実行できます:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

各タプルが個別のテストケースになります。説明文の `{0}`, `{1}` は実際の値で置換されます。

プロパティベーステスト

`@property` を使ってランダム生成された入力でテストできます:

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

テストは `count` 回ランダム値で実行されます。失敗時は反例が表示されます。

制件事項

- `describe` のネストはサポートされていません
- `before_each` / `after_each` はサポートされていません
- オーバーロード関数および `@native` 関数はモックできません

[← 前: 契約による設計](#)